Kafka Lag Analyser

A Plug-and-Play Fault Detection Module for Apache Kafka

CS349 Project Report

Hari (23B0907) Tejas (23B0932) Avinash (23B1064) Rishi (23B1081)

May 2026

Overview

The Kafka Lag Analyser is a plug-and-play monitoring module that can be attached to any existing Apache Kafka deployment with minimal configuration changes. Once running, it continuously monitors the cluster and surfaces pre-emptive, high-signal alerts when it detects one of five known failure patterns.

The design goal is to give a Kafka administrator actionable warnings *before* a failure cascades into user-visible downtime. Rather than relying on raw metric thresholds, the system uses trained machine learning models to interpret patterns across many metrics simultaneously and report a maximum of four concise, context-rich alerts per incident. It names the affected consumer group or topic alongside the exact signals that triggered the detection.

Integrating the module into an existing deployment requires exposing the Kafka broker's JMX metrics via the standard Prometheus JMX Exporter agent. This is typically a single JVM argument added to the broker's startup configuration. The scraper and analyser services are then started alongside the cluster using the provided Docker Compose file.

Motivation

A production Kafka cluster emits thousands of metrics: per-topic byte rates, per-partition log offsets, request latencies broken down by percentile and request type, consumer group states, replication lag, thread pool utilisation, and more. In practice, operators set threshold alerts on a handful of these and watch dashboards for the rest. This approach has two well-known problems. First, a single fault condition often shows up as correlated anomalies across dozens of metrics at once—an operator looking at individual graphs has to manually connect the dots under pressure. Second, threshold-based alerts fire constantly on healthy clusters with variable load, creating alert fatigue that causes real problems to be missed. The core idea behind this project is to push the complexity of interpreting raw metrics into machine learning models, so that the operator only ever sees the output: a small number of labelled fault events, each pointing to the specific entity (consumer group, topic, or broker) that is affected. The human monitors the result of the model, not the metrics themselves.

Product Specification

Fault Classes

The system detects five fault classes, each corresponding to a common operational problem in Kafka deployments:

Class 1. Rebalance Loop : A consumer group enters a continuous cycle of rebalancing without ever stabilising. This typically happens when session timeouts are misconfigured, when consumers crash during assignment, or when partition topology changes faster than consumers can keep up.

Class 2. Broker Saturation : The broker's CPU or I/O resources are exhausted. Produce requests begin to queue, request handler threads become idle waiting for disk, and throttle times spike.

Class 3. Network Degradation : Elevated latency between the broker and clients, or between brokers in a replicated cluster. Under-replicated partitions appear and inter-broker replication lag grows.

Class 4. Partition Skew : Write traffic is concentrated on a subset of partitions of a topic while others are idle. This causes uneven storage growth and hot-spot latency for the partitions absorbing all traffic.

Class 5. Slow Consumer : A consumer group is processing messages slower than producers are writing them. Lag grows monotonically rather than fluctuating around a steady state.

These classes were chosen so that they are operationally meaningful and also are disjoint in their causes. This makes it easier for the models to learn to distinguish between different fault types.

User Interaction Model

Figure 1 shows how an operator interacts with the system.

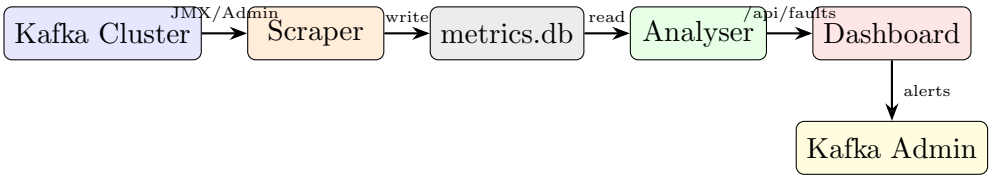

Figure 1: End-to-end data flow from Kafka cluster to the operator dashboard.

The operator opens `http://localhost:8000` in a browser. The dashboard auto-refreshes every three seconds. If all models report healthy, a green banner is shown. When a model fires, a red card appears naming the fault class, the affected entity (e.g. `consumer_group=my-app`), and a collapsible panel showing the exact metric values that triggered the detection.

System Architecture

The system is composed of four services, all orchestrated with Docker Compose:

1. **Kafka Broker** : KRaft cluster with the Prometheus JMX Exporter Java agent attached. The exporter filters and renames JMX MBeans into Prometheus-format metrics and exposes them on port `9101`.
2. **Scraper** : A Python service that polls the JMX exporter endpoint and the Kafka Admin API every 10 seconds. Each poll collects broker-level JMX metrics and per-partition consumer lag for all active consumer groups, then writes both into an SQLite database (`data/metrics.db`) shared via a host volume mount.
3. **Analyser** : A FastAPI service that wakes up every 10 seconds, loads the last 100 seconds of scrape data from the shared database, runs feature extraction, and feeds the resulting feature vectors into each trained XGBoost model. Any model predicting a fault (output = 1) produces a fault event that is stored in memory and served from `/api/faults`.
4. **Dashboard** : A static HTML/JS page served by the Analyser. It polls `/api/faults` every three seconds and renders the current fault state and a rolling history of the last 200 events.

Training Pipeline

Training is offline and separate from the runtime system. Figure 2 shows the pipeline.

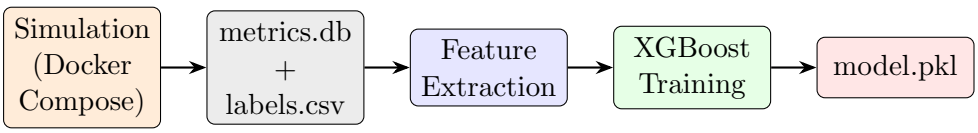

Figure 2: Offline training pipeline. One model is trained per fault class.

For each fault class, a simulation script spins up a local Kafka environment using Docker Compose (including a producer and a consumer) and deliberately induces the target fault condition. The scraper runs throughout, writing metrics into a `metrics.db`. Once the simulation ends, a `labels.csv` is generated that marks each scrape session as either `healthy` or the relevant fault class. The simulation methodology per class is as follows:

- **Rebalance Loop** - The simulation sends repeated consumer reconnection commands and induces session timeouts at 5-second intervals, then incrementally adds topic partitions to force continuous rebalancing.
- **Broker Saturation** - The broker container's CPU is throttled to 5% of a core using Docker resource limits, while the producer sends 5 MB random-byte payloads to simultaneously stress disk I/O.
- **Network Degradation** - Linux traffic control (`tc netem`) injects 400 ms of latency on the inter-broker network interface and 200 ms on client-facing interfaces, degrading both replication and client round-trip times. Note: the training data for this class was flawed due to missing JMX exporter rules, so the model trained on it is not valid.
- **Partition Skew** - The producer is configured to put all writes to a subset of partitions.

- **Slow Consumer** - The consumer's processing rate is artificially throttled to 20% of the producer's write rate.

Features are extracted from the raw time-series data by computing rolling statistics (slopes, standard deviations, coefficient of variation) over a window. This transforms point-in-time snapshots into trend-aware signals. Each model is then trained using XGBoost with five-fold stratified cross-validation and class-balanced sample weights. The trained models are serialised as `.pkl` files and mounted into the analyser container at runtime.

Inference Pipeline

Figure 3 shows the runtime inference pipeline.

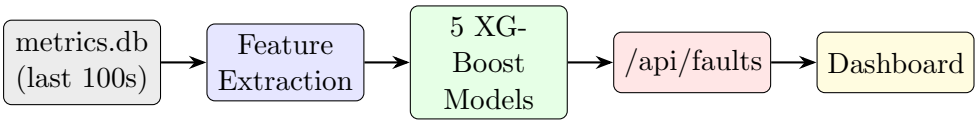

Figure 3: Runtime inference pipeline, repeated every 10 seconds.

Model Performance

Table 1 shows the mean cross-validation F1 score (fault class = 1) for each trained model.

Fault Class	CV F1 (fault)	Note
Slow Consumer	0.98	
Partition Skew	0.99	
Rebalance Loop	0.87	
Broker Saturation	0.78	
Network Degradation	0.11	Training data issue (see below)

Table 1: 5-fold stratified cross-validation F1 scores per fault class.

Four of the five models perform well. The network degradation model scores poorly because the simulation data for that class contained near-zero values for all latency and throughput metrics — the JMX exporter rules for several key metrics (`RemoteTimeMs`, `ResponseSendTimeMs`, throughput counters) were missing from the exporter configuration at the time of data collection, meaning the broker never published those metrics and the scraper stored zeros. The model has nothing to split on. Retraining after fixing the exporter configuration is the immediate remediation.

Testing

The system was tested on a demo cluster whose workload can be varied using the same configurable knobs as the training simulations. To evaluate a given fault class, the corresponding fault knob is activated and the dashboard is observed to confirm that the correct fault card appears within one to two analysis cycles (10–20 seconds). Returning

the knob to the healthy state should cause the alert to clear on the next cycle. This procedure was carried out for each of the four well-trained fault classes.

The demo cluster was also run in a sustained healthy state for an extended period to check for spurious alerts. No false positives were observed during this period for the slow consumer, partition skew, and rebalance loop models. The broker saturation model occasionally fired false positives under normal load spikes, which is consistent with its lower F1 score and the relatively coarse features available for that class.

Use of AI

GitHub Copilot was used extensively throughout this project. The primary use was code generation: Copilot was prompted with descriptions of what each module needed to do (e.g. "write a FastAPI endpoint that loads recent scrape data from SQLite and runs XGBoost inference"), and the generated code was reviewed, corrected, and **integrated** by the us. Architecture decisions were made by the us and then implemented with Copilot assistance.

Copilot was also used for debugging.

Future Work

- **Network degradation retraining.** The most urgent next step is fixing the JMX exporter configuration for the missing metric rules and re-running the class 3 simulation to produce a valid training dataset.
- **Online model updating.** Models are trained offline on simulations. A mechanism to periodically retrain on data collected from the live cluster would allow the models to adapt to deployment-specific traffic patterns.
- **Root-cause ranking.** When multiple models fire simultaneously, there is currently no ranking of which fault is most likely primary. Adding a confidence score to each fault event (e.g. using XGBoost's `predict_proba`) would allow the dashboard to surface the highest-confidence alert first.